Chapter 13
Machine Learning-Enhanced Analysis and Design for Trustworthy Integrated Circuits



Kaixin Yang, Yinghua Hu, Dake Chen, Chunxiao Lin, Yang Yi, Peter A. Beerel, and Pierluigi Nuzzo

13.1 Introduction

Nowadays, integrated circuit (IC) manufacturing has become a collaborative process, with many companies participating in different stages, including design, fabrication, assembly, test, and distribution, due to the cost efficiency and diverse expertise offered by different companies. However, such an involvement of third-party players brings concerns regarding potential security threats, such as sensitive data leakage, intellectual property (IP) theft, design tampering, and counterfeit designs [1–3]. Establishing trust across the supply chain usually requires appropriate measures to ensure a design's security, dependability, and resilience.

As shown in Fig. 13.1, threats that affect the hardware implementation of a system can be mainly classified into five categories: (a) malicious modification, (b) IP piracy, (c) side-channel analysis, (d) reverse engineering, and (e) counterfeit

K. Yang (✉) · D. Chen · P. A. Beerel
Department of Electrical and Computer Engineering, University of Southern California, Los Angeles, CA, USA

Y. Hu
Synopsys, Inc., Mountain View, CA, USA
e-mail: yinghuah@synopsys.com

C. Lin · Y. Yi
Department of Electrical and Computer Engineering, Virginia Tech, Blacksburg, VA, USA
e-mail: chunxiaol@vt.edu; yangyi8@vt.edu

P. Nuzzo
Department of Electrical and Computer Engineering, University of Southern California, Los Angeles, CA, USA

Department of Electrical Engineering and Computer Sciences, University of California, Berkeley, Berkeley, CA, USA
e-mail: nuzzo@eecs.berkeley.edu

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2025

A. Iranmanesh, H. Sayadi (eds.), *AI-Enabled Electronic Circuit and System Design*,
https://doi.org/10.1007/978-3-031-71436-8_13

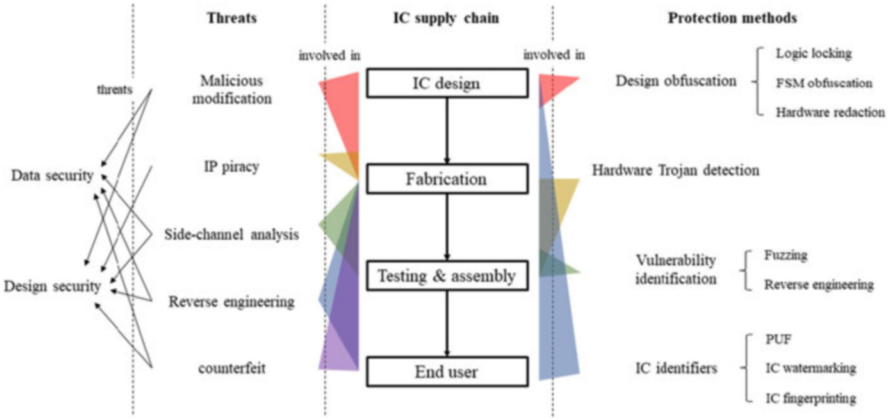


Fig. 13.1 Typical threats and possible protection methods across the IC supply chain

[4], which jeopardize the security of the hardware design itself as well as the data fed into the design. Over the years, various techniques have been proposed to mitigate risks and protect confidential information for hardware designs [3–11]. Design obfuscation techniques aim at preventing unauthorized access to the correct functionality of a digital design by either introducing additional circuitry, controlled by a key, or by replacing the original circuitry with a reconfigurable block. Hardware Trojan detection methods monitor the behaviors of the design under test (DUT) for designated test patterns to reveal malicious modifications to the original design. Vulnerability identification tools aim to broadly identify the type and location of potential vulnerabilities by simulating and analyzing the functionality of a circuit. Finally, IC identifiers can be inserted into the circuit to identify the ownership and usership. However, despite the many existing algorithms and strategies, trustworthy hardware design may remain a challenge. At design time, existing obfuscation methods often utilize heuristic evaluation metrics and rarely rely on formal foundations to define their security [12, 13]. During testing, vulnerability identification tools usually leverage heuristic methods to generate test patterns [14]. These difficulties are then compounded by scalability, both at the design time and test time, due to the size of modern integrated systems [15]. To summarize, conventional hardware design methods (1) usually have limited capability in capturing patterns in complex systems where nonlinear and high-dimensional relationships are involved, (2) often struggle to adapt to different scenarios, (3) may require a substantial amount of manual effort for specific applications.

Similar challenges have already appeared in many other domains, and some existing approaches attempt to find solutions by integrating machine learning (ML) with conventional methodologies due to ML's advantage in pattern recognition, task automation, and versatility. On the other hand, ML models may exhibit unpredictable behaviors and are difficult to interpret [16, 17]. More conventional methods can then be used to rectify their outcomes. For example, simulation can

provide precise results if a physical model is known. In contrast, ML requires less understanding of the model but can give accurate predictions if provided with sufficient and unbiased data. Therefore, ML can be used to calibrate the parameters of a simulation model to improve the estimation accuracy, while simulation can assist in generating high-quality data for ML techniques [18]. In the context of computer vision (CV), for example, deep learning (DL) has been widely used in processing digital images, and its integration with more conventional CV techniques can achieve a better trade-off between performance and cost in many situations [19].

Given the success of hybrid, data-driven, and model-based design methods in other domains, this chapter explores the possibility of using ML to help build trustworthy hardware systems. A few works have already leveraged ML to improve the performance of conventional tools and address previously unsolved problems. We review the application of ML techniques in the context of hardware protection methods through the whole IC supply chain. We discuss their advantages and limitations as well as the benefits of integrating them with conventional design flows. Finally, we introduce promising directions for future research.

13.2 Design-Time Methods and Tools: Circuit Obfuscation

In the last few years, many researchers have been actively delving into research about building trustworthy hardware design, as well as formal metrics and methods to effectively and quantifiably evaluate the security level of the VLSI circuit [12, 20–25]. Modern design and manufacturing of ICs heavily rely on a global supply chain, raising concerns about the security and protection of hardware intellectual properties (IPs). To address these concerns, strategies that can obfuscate the original designs and can unlock the correct functionality only when correctly configured are proposed. Among them, a promising method called logic locking (LL) can be employed to safeguard hardware IPs from untrusted foundries. Typically, LL approaches involve incorporating additional logic that enables the introduction of secret keys. When an incorrect key is applied to the locked circuit, the circuit outputs produce incorrect results, effectively restricting access to the correct circuit functionality. Numerous LL approaches are proposed to provide protection for both combinational and sequential circuits, along with dedicated techniques for scan chains. Ideally, certain properties of logic locking, such as the correct key, should be kept as secrets from attackers. However, the implementation of logic locking may depend on these secrets. The unwanted correlation between functional/structural properties of the locked circuits and the secrets opens up a new attack surface where novel machine-learning approaches can learn from these correlations and make informative predictions on the secrets.

In this section, we focus on introducing existing applications of ML techniques to assist the attack on logic locking as a security evaluation method. We will also briefly summarize a few techniques that attempt to protect the locked design from being attacked by removing the signatures leveraged by ML. In the end, we provide a discussion on the benefits brought by ML and the potential future directions.

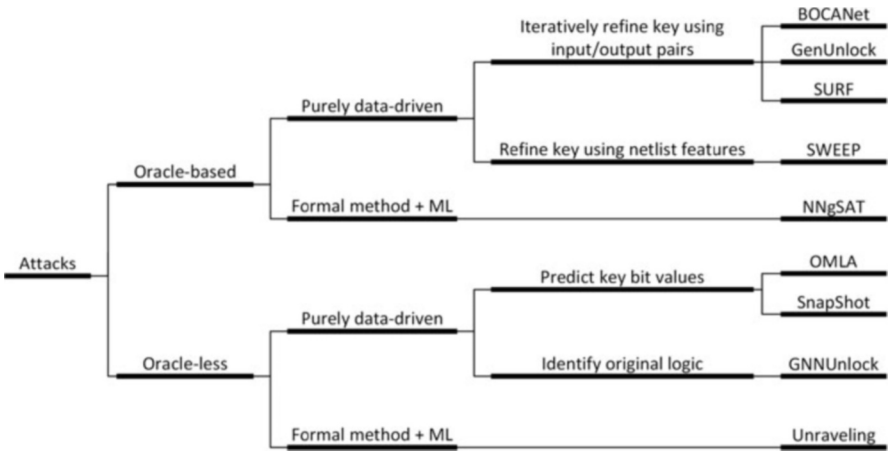


Fig. 13.2 Different types of ML-assisted attacks on logic locking techniques

13.2.1 ML-Assisted Attacks on Logic Locking (Fig. 13.2)

Attacks on logic locking techniques have been widely developed and used as a way to evaluate the performance of those defense techniques. The runtime or the key value prediction accuracy of successful attacks on different techniques can represent their security level under corresponding attack surfaces. Generally, there are two types of attacks for LL. The first one is oracle-based; it assumes that the attacker can access a functioning unlocked circuit's primary inputs and outputs. This category includes attacks based on satisfiability checkers (SAT) [26], sensitization [27], removal and bypass [28, 29], functional analysis [30], and GF(2) linear algebra [31]. In contrast, oracle-less attacks exploit structural signatures within the locked netlist only, associated with specific defenses. Notable examples include OMLA [32], GNNUnlock [33], Snapshot [16], among others. In the rest of this subsection, we are going to introduce several typical attacks leveraging ML techniques.

13.2.1.1 Oracle-Based Attacks

BOCANet [34] is one of the oracle-based attacks exploring the use of deep learning techniques, specifically Deep RNN, to break logic locking techniques. A Deep RNN model is trained using input/output pairs, and then the trained model is used to predict and update the key value iteratively according to the mean square error of the newly generated outputs. In this technique, the circuit is treated as a black box, and only the input/output pairs are used for refining the key value. GenUnlock [35] proposes an approach based on genetic algorithms (GAs) and particle swarm optimization (PSO) to identify the correct keys for locked circuits. It generates input vectors and queries the active IC to collect corresponding outputs as a training

dataset. In the key evolution phase, a set of keys are instantiated as individuals in the population. GenUnlock performs circuit fitness evaluation, population selection, crossover, and mutation operations to evolve the population and search for the correct keys. SURF [36] is another oracle-based attack that leverages ML and refines the key value iteratively. It takes one step further as it utilizes the structural information of the netlist as well. SURF starts a correct key exploration process with a seed key generated by a machine learning-based attack called SAIL (structural analysis-based IC Locking) [37]. SAIL predicts structural changes introduced by resynthesis and reverts them to provide insight into the design before resynthesis. This information can be used for subsequent attacks, such as key retrieval or reverse engineering. The SURF algorithm starts with a seed key from SAIL and iteratively refines it by flipping key bits and measuring the output corruption. If the bit flip results in less corruption, it is kept. This process continues until the correct key or an approximate key is determined. BOCANet, GenUnlock, and SURF find candidate key values as the starting point and iteratively refine the key value. They all take the functional information as the refinement metrics, which ensure that the final predicted key can retrieve the correct functionality or approximate functionality of the original design. However, since they are all purely data-driven, they cannot guarantee the correctness of the final key they find for any new design when terminating their algorithms.

Additionally, instead of only using output information for determining the correct key, SWEEP [38] propagates constant values of 0 and 1 from key ports to collect circuit features from synthesis tools, which can disclose key bit values by analyzing discrepancies in characteristics. The features include metrics such as area reduction, number of edges, power consumption, and gate count. The correlation between these design features and the key values differs for each locking scheme, and SWEEP exploits vulnerabilities in different locking techniques. For example, the MUX-based TOC locking technique maximizes the Hamming distance between the correct and incorrect routes. Applying an incorrect key value will result in a large area reduction and changes in other design features. Therefore, the impact of key value on the correlated netlist features can be learned, and then the correct key value for any locked design can be estimated according to the obtained feature values. Similar to the techniques mentioned above, SWEEP is also data-driven and thus faces the same challenges.

Moreover, NNgSAT [39] is a neural network (NN)-guided SAT attack on circuits that are locked with or contain complex structures, making them hard to solve using traditional SAT solvers. The goal of NNgSAT is to speed up the SAT attack by using a trained neural network as a classifier to predict the satisfying assignment. NNgSAT utilizes a message-passing neural network (MPNN) with an iterative structure. It consists of two stages: in the first stage, each clause receives messages from its neighboring literals and updates its embedding based on the current embeddings. In the second stage, each literal receives messages from its neighboring clause and its complement and updates its embedding based on the current embeddings. This iterative process continues for a certain number of iterations and finally votes for the literal values that can solve the SAT problem with high confidence. Different from

previous techniques, NNgSAT embeds ML techniques into the conventional SAT attack flow, which accelerates the assignment prediction step. It combines formal methods with ML techniques, which ensures the correctness of the final result as well as reduce the runtime.

13.2.1.2 Oracle-Less Attacks

OMLA [32] utilizes graph neural networks (GNNs) to extract key bit values from locked netlists without the need for an oracle or prior knowledge of the synthesis procedure. OMLA addresses the limitations of traditional machine-learning (ML) models by leveraging the graph structure of netlists. It extracts small subgraphs for each key gate from the locked netlist and uses GNNs to learn representations of these subgraphs. The labels of the subgraphs correspond to the key bit values, allowing OMLA to classify and predict these values. SnapShot [16] is also a notable attack on logic locking that utilizes neural networks to directly predict a key bit value from a locked synthesized gate-level netlist without using a golden reference. The attack aims to immediately predict the key bit value based on the extracted locality (subgraph of gates) around each key gate. Compared to other techniques like SAIL, SnapShot offers several advantages. It can be applied to various locking schemes and netlist structures, not just XOR-based locking. It allows for direct prediction of key bit values based on post-resynthesized netlists, eliminating the need for access to the original synthesis tool used in the design flow. SnapShot also requires training only one model, simplifying the attack process. This method synthesizes designs to have fixed fan-in and fan-out sizes for each node, generates the locality feature vectors for each key gate in the extracted netlist graph, and trains MLP and CNN models to predict each key bit value. Both OMLA and SnapShot predict the key bit value based on the structural information of the key port surroundings. According to their experimental results, both techniques can achieve high prediction accuracy for each single key bit. However, challenges still exist for them. For example, when the key length is large, the overall prediction accuracy for the whole key string may not be high enough. Also, these two techniques do not take the functional information into consideration, which means key values with high accuracy cannot guarantee the retrieved netlist functionality is similar to the original one.

Another oracle-less attack using the ML technique is GNNUnlock [33]. The goal of GNNUnlock is to automatically detect and remove the protection logic added by LL schemes, thereby recovering the original functionality of the locked circuit. GNNUnlock first generates a dataset of locked benchmarks by locking each benchmark multiple times with randomly generated key values and different key sizes, resulting in an extensive training set. The training process is not aware of the correct key values. Next, the locked benchmarks are transformed into graphs using a netlist-to-graph transformation. Each node is associated with a feature vector that describes its important characteristics, such as its in-degree, out-degree, and connections to primary inputs and outputs. The GNN is trained to classify the nodes in the graph as either part of the protection logic or the original design

logic. GNNUnlock uses similar ML models and feature engineering with a different training target. The previous techniques train GNN models to classify the key input value as 0 or 1, while GNNUnlock trains GNN models to classify all nodes in the netlist to be part of the original design or not.

Moreover, an attack that leverages ML to pinpoint the structure signature in the latch-based logic locking (LBLL) circuit is introduced recently [40]. The attack is a two-phase attack on latch-locked circuits using a combination of deep learning, Boolean analysis, and integer linear programming (ILP). The goal of the attack is to identify the secret key used for latch locking and fully disclose the correct functionality of the circuit. In the first phase of the attack, machine learning is used to detect and remove logic decoy latches from the locked circuit. A random forest classifier is trained to identify these decoy latches based on extracted sequential graphs and feature vectors. The circuit is then simplified through constant propagation. In the second phase, the remaining latches in the circuit are classified using a combination of machine learning and ILP. Ground-truth labels are used to train a second classifier that distinguishes primary/secondary latches from delay decoys. The softmax outputs of this classifier are used as coefficients in the ILP objective function, which aims to maximize the probabilities of latch classifications. The ILP solver is configured to find the closest legal key to that identified by the machine-learning classifier. This technique is another example of combining ML techniques and formal methods. Similar to NNgSAT, this method uses ML models to obtain good prediction results to facilitate the ILP solving process.

13.2.2 Defenses Against ML-Assisted Attacks on Logic Locking

Defenses against ML attacks are also starting to appear. Since ML-based attacks mainly learn the correlation between the target, such as key value, and the sensitive information, such as the structural features, existing defenses against ML-assisted attacks focus on reducing this kind of correlation. There is not as much work for defense compared to attack, which indicates a great potential to further explore efficient defenses in the future.

UNSAIL [41] is a defense scheme designed to confuse oracle-less machine-learning attacks on logic locking. UNSAIL aims to inject adversarial samples during the learning process of ML models used in attacks like SAIL. It does this by introducing identical subgraphs in the locked netlist that can be classified as either changed or unchanged, confusing the ML models. By injecting this conflicting data, UNSAIL undermines the training efforts of the ML models and induces flawed inferences. The effectiveness of UNSAIL is evaluated against the SAIL attack, as well as other attacks like SWEEP and redundancy attacks. The results show that UNSAIL effectively reduces the accuracy of ML-based attacks on logic locking. It confuses the ML models used in the attacks and makes it difficult for them

to accurately predict changes in the locked netlist. UNSAIL also demonstrates resilience against other attacks and maintains a high level of security. UNSAIL provides a promising defense against oracle-less ML attacks on logic locking, enhancing the security of integrated circuits and protecting them from unauthorized access and reverse engineering.

DECOR [42] is a recent technique that enhances existing logic locking techniques against ML-assisted attacks. This method focuses on decorrelating the design structure from the correct key value, since the oracle-less attacks heavily leverage the structural information to predict the key value, such as OMLA and SnapShot. By modifying the locked design functionality and introducing more correct key strings, a single key bit can have different correct values in different key strings. Meanwhile, different key bits that have opposite correct values can also share similar structural features. The validation results exhibit that the proposed approach confuses the OMLA attack and reduces the average key prediction accuracy by more than 30% and 10% after enhancing two logic locking techniques, XOR-based insertion and SARLock, respectively.

13.2.3 Discussion and Future Directions

As introduced in the previous subsection, existing ML techniques leverage either functional or structural information to assist in simulation result prediction, key value prediction, assignment exploration in an SAT problem, node classification, etc. The ML techniques can be integrated into the conventional attack flow as a preprocessing or post-processing step, or can be used to train a model with the information provided by the conventional methods and analysis. The combination of ML techniques and conventional methods can rectify the prediction results from the ML models to achieve higher accuracy, as well as reduce the effort and search space of conventional methods.

We notice that neural networks, especially GNN, have exhibited good capability to capture structural information. Since a netlist can be naturally converted into a graph by treating gates as nodes and connections as edges, the aggregation function of GNN can gather the information around a node, generating a locality signature. The information from the node surroundings helps improve the property prediction and node classification accuracy, as shown in OMLA and GNNUnlock. We expect to see more GNN-based techniques appear in related areas, aiding the conventional flow and improving performance.

Recently, large language models (LLMs) have become attractive, and researchers are exploring the possibility of applying LLMs in RTL designs [43, 44]. The LLMs are usually trained with a massive amount of data and thus have the capability to provide solutions for commonly used designs promptly. However, for some specific or constrained designs, LLMs may have limitations since they are heavily dependent on statistical patterns and have limited capability of rule-based reasoning. To build a trustworthy hardware design, potential vulnerabilities should

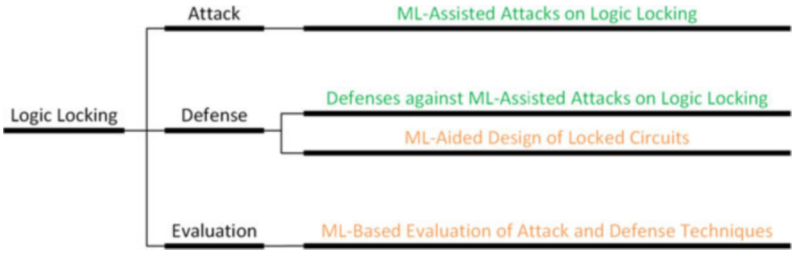


Fig. 13.3 Interesting topics related to ML-assisted logic locking

be considered, which means a design with specific security-aware constraints, such as low correlation between correct key values and the structure. In this case, we believe rule-based conventional design methods could help LLMs implement risk-free hardware designs in the future.

Furthermore, we mainly introduce the existing ML-assisted attacks on logic locking, as well as the defenses against these attacks. Except for them, we believe there are also other interesting topics that can be further explored, as listed in the Fig. 13.3. For defense methods, integrating ML techniques into the logic locking automation design flow can be another interesting direction. ML techniques can help make good decisions about the locking location, configuration, and choices of locking primitives. However, challenges exist, such as how to quantitatively define a good locked netlist for ML models to learn and design. For ML-based evaluation of attack and defense techniques, a recent paper [45] starts to discuss possible ways to estimate the SAT attack runtime given a locked netlist without running the actual attacks. Using ML-based evaluation methods can either save the evaluation runtime or provide a solution when a concrete evaluation metric is hard to explicitly express.

13.3 Test-Based Methods and Tools

Due to the distributed ICs production process, untrusted third-party companies can be involved in the supply chain. Potential adversaries from untrusted third-party companies can maliciously tamper a design by inserting hardware Trojans (HTs), which will be triggered under certain conditions, and cause multiple potential threats, including confidential information leakage, functional corruption, and even permanent fatal damage to the chip [46]. Therefore, the hardware Trojan detection mechanism is essential in realizing trustworthy IC design. Conventional HT detection methods can be classified into three main types based on the detection mechanism: (1) logic testing, (2) side-channel analysis (SCA), and (3) reverse engineering [6]. All these three types of HT detection methods share a few common challenges, including the requirement of a golden model, the scalability issue, and high cost. Recently, the success of machine learning (ML)-based techniques in many

domains has inspired research on detecting HTs to improve conventional algorithms using ML-based techniques [46–49].

ML-based techniques can assist existing HT detection methods in many aspects. Logic testing aims at generating test patterns that can trigger Trojans, where reinforcement learning (RL) may be used to improve the efficiency to find a minimal set of input patterns that can activate Trojans with high coverage [50–52]. SCA detects HTs by identifying differences between side-channel measurements associated with HT-infected and HT-free designs, which can be enhanced by support vector machines (SVM) to reveal unseen signature [53]. In addition to detection techniques during the test process, there are also ML-based techniques for detecting HTs during runtime [54, 55]. Meanwhile, golden model-free HT detection is another attractive research topic. Except for some methods mentioned above, other techniques [56–59] also emerge due to the possible unavailability of a golden model and the high cost of golden model-based techniques. These techniques leverage the differences in certain features, such as side-channel properties, controllability, observability, and signal correlation, between HT-infected and HT-free designs or signals to train classifiers, and finally classify unseen designs.

In this section, we focus on applying ML techniques in logic testing and SCA-based HT detection methods. Reverse engineering can be treated as a more general technique of design vulnerability identification and will be discussed later.

13.3.1 ML-Enhanced Hardware Trojan Detection (Table 13.1)

A few works [60, 61] introduce early attempts to integrate ML methods into hardware Trojan detection techniques. These works model hardware designs as graphs and propose a few graph properties to compose the feature vectors associated with each net in the circuit. SVM and random forest (RF)-based methods are adopted to classify whether or not a net is among the ones constructing hardware Trojans. The proposed features include the number of each circuit element within a certain depth inside the input and output cones of the target net. Compared with a previous method leveraging signal correlation between Trojan nets and normal circuits, ML-based techniques may outperform the previous work regarding true positive rate (TPR) and the final precision.

AdaTest [50] is proposed to address the effectiveness, efficiency, and scalability challenges of conventional logic testing-based HT detection methods. In general, the proposed method defines HT-related reward functions and then utilizes reinforcement learning to generate high-quality HT detection test patterns iteratively. The transition probability of each signal is obtained through random logic simulation to locate rare nets, which are more probable to be leveraged by an adversary to trigger HTs. The testability values, including controllability and observability, are also calculated during the circuit profiling step of AdaTest. The transition probability and testability work as the rewards for the reinforcement learning model to generate candidate test patterns. According to the evaluation results, AdaTest can achieve

Table 13.1 Brief introduction to the ML-based hardware Trojan detection methods

HT detection method	ML technique	Information by leveraged ML	Contributions of ML	How combined with conventional methods
Trojan feature generation [60, 61]	SVM and RF	Structural information in the fan-in and fan-out cones	Reveal the correlation between the HT-affected logic and the structural features to improve detection precision	Conventional methods extract and generate structural features for training the ML model
AdaTest [50]	RL	Signal behaviors, including transition probability and testability	RL models are trained and used to select high-quality test patterns for detecting HTs	Transition probability and testability values are calculated during the conventional circuit profiling step for calculating the rewards during RL-based test pattern generation
GNN4TJ [62]	GNN	DFG of the hardware design	Identify the existence of HTs based on the different data flow behaviors	The DFG and corresponding features are generated using conventional methods for training the GNN model
Power signature analysis [53]	SVM	Power traces and PCA results	SVM captures the power signature difference between HT-infected and HT-free designs	PCA on the collected power traces extracts the signatures to feed the ML model

high Trojan coverage with fewer input patterns across various benchmarks. Similar to AdaTest, another work [51] uses RL to update test patterns by flipping bits instead of adaptively sampling from candidate test patterns according to their reward values. Furthermore, DETERRENT [52] focuses on maximizing the rare nets that can be covered by one test pattern so that each pattern is more effective in detecting HTs, and thus, the overall test pattern set could be minimized.

GNN4TJ [62, 63] proposes to utilize graph neural network (GNN) model to classify hardware Trojan-free and hardware Trojan-infected design by learning design behavior features from their data flow graphs. GNN4TJ addresses the challenge that (1) the golden reference, which is the design without any malicious modification, might not be available during the testing phase for detecting hardware Trojans, and (2) the type of the potentially inserted Trojan is unknown. The proposed method generates a data flow graph (DFG) for the given hardware design and then extracts graph-level features from the DFG. After that, the GNN model will be trained with a collection of HT-free and HT-infested designs generated using various types of hardware Trojans and employed to detect the existence of Trojans in any given design later. GNN4TJ leverages the fact that introducing hardware Trojans can bring uncommon data flow behavior into a design. GNN models can learn the behavior patterns from the training data and thus distinguish HT-infected designs based on their data flow behavior without any knowledge about their golden reference.

Another work [53] explains how ML-based techniques help improve SCA-based HT detection. This work applies principal component analysis (PCA) on the collected power traces and then provides generated power signatures for training an SVM model to distinguish between IPs with and without HTs. Compared with conventional methods, the ML-based approach helps eliminate the effects of parameter variations. Similarly, SVM models are also helpful in runtime HT detection [54, 55]. The trained models can identify the Trojan's behavior based on the collected runtime information. These approaches are more time-efficient than conventional techniques.

13.3.2 Discussion and Future Directions

Nowadays, hardware design is becoming even larger and more complex. Detecting HTs, which can be an extremely small portion of the design, will be more challenging regarding time efficiency and cost. ML-based approaches provide many opportunities to improve conventional HT detection techniques. They can help identify HTs from different perspectives, such as the circuit logic, structure, and physical properties, while they are usually efficient during the inference phase if the models are trained well. In most situations, ML models only rely on the properties of the target design itself without the necessity of a golden model, which also brings more flexibility.

However, ML techniques can also be applied oppositely. ML models can be used to learn from the features of hard-to-detect HTs and then automatically insert high-quality Trojans into the design [64]. This will bring more challenges to the existing HT detection techniques. Similar to logic locking techniques, new HT insertion techniques can focus on the design signatures leveraged by current detection methods, and hide or reduce the signature difference of inserted Trojans compared to the normal design. Recent research exhibits the possibility of utilizing the large language model (LLM) and natural language processing (NLP) to detect HTs as well [65]. They generate the natural language description of the hardware design and use GPT-4 to understand and analyze the contextual and structural intricacies. The experimental results show that GPT-4 can achieve detection performance comparable to traditional ML algorithms. This research opens another direction for ML-based HT detection and needs further exploration. For example, a fine-tuned LLM may perform better than the general GPT-4 model, and different information about the design can be described using natural language and fed into the LLM model to improve HT detection.

Therefore, we believe ML-enabled techniques can help overcome some existing challenges in HT detection and raise the design testing and verification efficiency to a higher level.

13.4 ML-Enhanced Vulnerability Identification

As the functionality of hardware design is becoming increasingly complex, detecting and locating vulnerabilities inside the design is even more complicated [66]. it is unrealistic to conduct exhaustive testing before deployment. Therefore, intelligent vulnerability identification becomes critical to avoid potential vulnerabilities introduced by careless designs. Meanwhile, the globalization of the IC supply chain also gives the adversaries chances to alter the design maliciously. Identifying the vulnerabilities brought by the malicious modification is also important [67]. In this section, we want first to introduce hardware fuzzing, which has been an attractive hardware verification technique in the last five years and contains many potential opportunities to have ML techniques involved. In the second part of this section, we want to emphasize the important role of hardware reverse engineering (RE) within the vulnerability identification flow and how ML facilitates the conventional RE methods.

13.4.1 Hardware Fuzzing

In a software development cycle, detecting potential vulnerabilities of the software is a crucial step to ensure the safety and security of the application that runs the software. In general, software vulnerabilities can be detected by two analysis

methods, namely, static methods [68–70] and dynamic methods [71–73]. Static analysis methods involve analyzing the source code of the software application without directly running the code. For example, one of the commonly seen software vulnerabilities is buffer overflow [74], where the code attempts to write to a predefined buffer with data whose size is larger than what the buffer can hold. A static analysis tool can go through the source code and identify suspicious buffer write operations that fall into this category. Static analysis methods are often known for their efficiency in scanning the source code. However, not all software vulnerabilities are straightforward to be seen in the source code; some can only manifest in a specific runtime environment. For these vulnerabilities, dynamic analysis methods come into play where certain input stimuli to the software can be generated to "trigger" the vulnerability and cause abnormal functional behavior, such as unexpected function output and runtime crash, during code execution. Fuzzing, as one of the most prominent dynamic methods to detect software vulnerabilities, has received much attention in both academia and industry [75, 76].

At the high level, a fuzzing tool, also called a fuzzer, operates through a flow comprising several key components: seed generation, input mutation, execution monitoring, and vulnerability analysis. Initially, a fuzzer generates inputs or "seeds" that are fed into the target software. These seeds undergo systematic mutations to produce a wide spectrum of test cases. The mutated inputs from the seeds are then executed, and the software's behavior is closely monitored for anomalies, such as crashes or memory leaks, revealing potential vulnerabilities. In this sense, the mechanism of fuzzing is similar to that of automatic test pattern generation (ATPG) [77] in chip testing, where test patterns are incrementally generated that can trigger circuit faults and propagate them to the primary output. Unsurprisingly, there is an ongoing effort to extend fuzzing to detect vulnerabilities in hardware, by converting the RTL code of the hardware into software [7]. Despite the efficacy of traditional fuzzing, its performance can be heavily limited by the quality of the starting seeds, which requires fine-tuning by experts with domain knowledge. Recently, various machine-learning (ML) techniques have been applied to several components of fuzzing, aiming to further improve the quality and speed of fuzzing.

In this section, we aim to provide a comprehensive view of previous efforts on applying ML techniques to fuzzing. In the end, we provide our insights on the future directions of integrating ML techniques for the specific application of hardware fuzzing.

13.4.1.1 Traditional Fuzzing

A fundamental fuzzing tool is structured around four core components: test case generation, code execution, program monitoring, and vulnerability analysis.

Test case generation This initial phase often requires a seed file that adheres to the target software's input format. Incompatibility with this format can result in immediate rejection during the program's entry point, causing inconsequential

failures. From this seed file, test cases are derived using three primary methods: mutation-based [76, 78, 79], generation-based [80, 81], and evolution-based strategies [82]. Mutation-based approaches are typically employed when there is limited knowledge about the software, with new test cases crafted by applying predefined mutation rules, such as bit flipping, byte substitution, and data injection. A well-known example is AFL (American Fuzzy Lop), which is renowned for its efficient algorithm that smartly mutates inputs based on code coverage feedback, quickly uncovering a wide range of bugs [76]. Conversely, generation-based methods are utilized when there is an understanding of the software's input requirements, enabling the creation of varied test cases that inherently satisfy the input criteria without the need for a seed file. Grammar-based fuzzing is a classic example of this method, where the fuzzer generates inputs based on the grammatical rules of the expected input format [80]. The mutation and generation strategies, while effective, operate somewhat blindly, without insights into the utility or redundancy of the test cases they produce. To overcome this, evolution-based methods were developed. These methods utilize feedback from the execution of earlier test cases to generate new ones with more targeted objectives, often employing genetic algorithms or evolutionary strategies. This approach allows the fuzzer to adaptively refine its test cases based on the results of previous iterations, significantly enhancing the effectiveness of the fuzzing process [82]. In the context of software fuzzing, a common aim is to enhance code coverage, encouraging the generation of test cases that collectively traverse more lines of code or unique execution paths [83].

Code execution In this stage, the generated test cases are executed in controlled environments, such as virtual machines or containers. This isolation is crucial for safety and for accurately monitoring the software's response to each input [84]. The speed of execution is a critical factor for efficient fuzzing, with faster execution allowing for more test cases to be processed in a given time, thus increasing the chances of uncovering vulnerabilities [85].

Program monitoring This component involves tracking the program's response to each test case using various metrics, such as crash reports and sanitizer outputs. The integration of sophisticated monitoring tools is crucial for effective fuzzing. These tools are designed to detect and log anomalies in program behavior, providing valuable data for identifying potential vulnerabilities [86, 87]. Studies have shown that specific monitoring techniques can significantly impact the rate of vulnerability discovery [14].

Vulnerability analysis The potential vulnerabilities identified during fuzzing are analyzed in this phase. Techniques to filter out false positives and prioritize true vulnerabilities are crucial. This analysis often involves distinguishing between exploitable bugs and noncritical anomalies, a process that can be as complex as the fuzzing itself. Post-fuzzing, static analysis methods are often employed to further scrutinize the findings, offering a deeper inspection of potential security issues revealed by the dynamic analysis [88, 89].

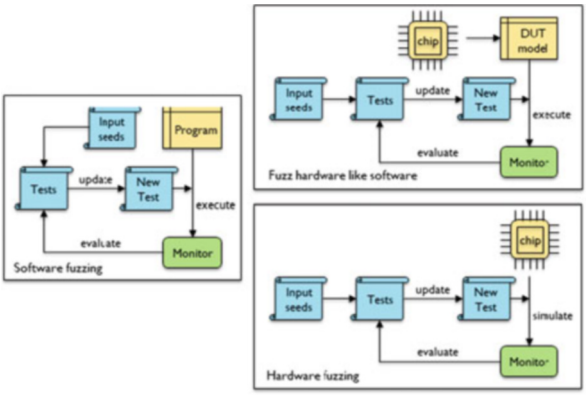


Fig. 13.4 Fuzzing for software and two basic implementations of fuzzing for hardware

Fuzzing for hardware has similar components and flow as fuzzing for software. As shown in Fig. 13.4, the first category of hardware fuzzing is to fuzz hardware like software. In this case, a software-based model will be created to capture the behavior of the hardware. This model will be executed with generated tests, and the monitor will check the behaviors to reveal potential vulnerabilities. On the other hand, another hardware fuzzing technique directly takes the hardware design and provides tests to run simulations. Based on the simulation results, the monitor will identify vulnerabilities inside the design. Due to the similarity between software fuzzing and hardware fuzzing, in the following subsections, we want to introduce the application of ML techniques in software fuzzing, and then discuss potential opportunities for hardware fuzzing.

13.4.1.2 Integrating ML into Fuzzing

As the complexity of software applications escalates, traditional fuzzing methods may face scalability issues, particularly when navigating deep state conditions and achieving extensive code coverage. For example, the selection of seed files is critical, directly influencing code coverage outcomes [90–93], and mutation operation choices further affect fuzzing efficacy [72, 94–98]. Machine learning (ML) has emerged as a powerful ally, enhancing the fuzzing process by refining components, especially during test case generation.

SmartSeed [91] uses an ML-based approach to learn from input files that have previously triggered unique crashes or new paths, and then uses this information to generate new seed files. Similarly, *Faster Fuzzing* [90] adopted two ML models, that is, Generative Adversarial Networks (GAN) and Long Short-Term Memory (LSTM), to enhance the American Fuzzy Lop (AFL) fuzzer's capabilities, yielding a higher rate of unique code path discovery. These advancements are echoed in other research efforts such as those by Cheng et al. [93] and Wang et al. [99]. The domain of sequential decision-making models, including Reinforcement Learning (RL) [72,

94–96], Q-learning [97], and Markov Chain [98], has been instrumental in refining mutation operation strategies. These models guide the fuzzing process through complex software landscapes by learning effective mutation patterns and adapting strategies based on feedback from previous fuzzing cycles. Collectively, these ML-enhanced techniques not only address the inherent limitations of traditional fuzzing methods but also pave the way for more intelligent and adaptive security testing frameworks capable of keeping pace with the rapid evolution of software systems.

Currently, not much work is being done on using ML techniques for hardware fuzzing. Although hardware fuzzing is similar to software fuzzing in terms of the flow, there are still many differences between the two domains, such as the definition of corruption. Most of the recent work focuses on how to implement the fuzzing in the context of hardware. On the one hand, since hardware and software fuzzing share a similar framework, all existing ML techniques utilized for software fuzzing can be migrated to the hardware domain to facilitate the seed input generation and the mutation operation for generating new input patterns. On the other hand, due to the complexity of hardware property verification, it is interesting to see how ML techniques can be leveraged to help generate meaningful hardware coverage points, as well as monitor and analyze hardware vulnerabilities.

13.4.1.3 Discussion and Future Directions

As we peer into the horizon of fuzzing methodologies, it becomes evident that the generation of test cases is only the prelude to the formidable challenge of their execution. The sheer volume of test cases necessitates a paradigm shift toward hardware-accelerated fuzzing solutions. By leveraging specialized hardware, such as GPUs or FPGAs, we can significantly reduce the execution time of test cases, enabling a more thorough exploration of the software's state space in less time.

Moreoever, machine learning has the potential to revolutionize the cost-function landscape of fuzzing. Traditionally, code coverage has been the de facto metric, under the presumption that higher coverage correlates with a higher likelihood of discovering unique bugs and vulnerabilities. However, this is not always the case. An ML-powered approach could entail learning a more nuanced cost function that prioritizes paths more likely to contain bugs, rather than just maximizing code coverage. For example, ML could analyze historical bug data to identify patterns or code features that are more bug-prone and adjust the cost function accordingly. ML could also be employed to predict the effectiveness of a test case before its execution, thus optimizing the allocation of computational resources. By learning from past executions, an ML model might estimate the probability of a test case revealing new vulnerabilities, allowing fuzzers to prioritize test cases with higher potential yields.

In the foreseeable future, we might witness the integration of ML models that can conduct a preliminary analysis of the code and suggest more focused areas for fuzzing, effectively creating a smarter, targeted approach to vulnerability discovery. This strategy could evolve into a semi-supervised learning model where fuzzing

tools learn and adapt from both their successes and failures, continually refining their test case generation and execution strategies.

As the intersection of ML and fuzzing grows ever more intertwined, the next generation of fuzzing tools will likely be characterized by their intelligence and adaptability, heralding a new era of software security testing that is both efficient and deeply insightful.

13.4.2 IC Reverse Engineering

Reverse engineering (RE) is an essential technique in the context of hardware security and assurance, which aims to retrieve confidential and critical design information about a product. Generally, RE consists of the following steps: (1) decapsulation, (2) delayering, (3) imaging, (4) annotation, and (5) schematic creation, organization, and analysis [9]. RE is a significant threat to the security of the integrated circuit (IC) design if it is illegally applied to a chip by an adversarial user. It also plays an important role in design verification and penetration testing. For example, RE is used for gate-level netlists to reconstruct the state transition graph for detecting potential modifications to the designs [17, 100] (Fig. 13.5).

ML techniques have been leveraged to accelerate and enhance the RE process. K-nearest neighbors (KNNs) and support vector machines (SVMs) are utilized to improve the performance of RE-based hardware Trojan detection approaches [9]. Graph neural networks (GNNs) increase the classification accuracy between state and data registers in a gate-level design [17], compared with previous heuristic methods [101–103], showing the advantage of ML-based techniques in providing effective classification models. The strength of ML is also leveraged in image-based RE [104].

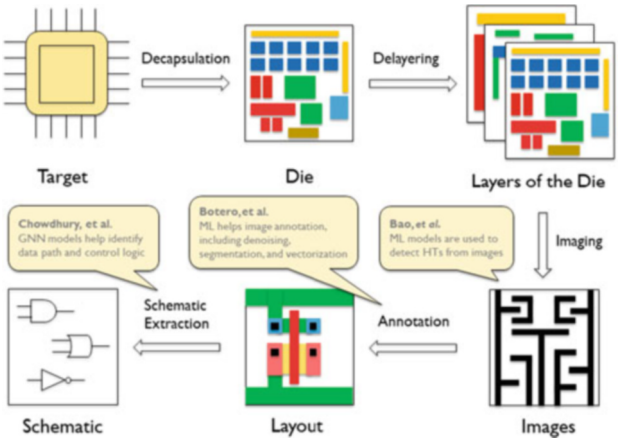


Fig. 13.5 IC reverse engineering steps

In this subsection, we want to introduce a few reverse engineering techniques with ML involved. We want to highlight the contributions of ML techniques at different steps of the reverse engineering flow. Three typical techniques are discussed below to explain how ML techniques help identify hardware vulnerabilities or how they help process the data to facilitate vulnerability identification.

13.4.2.1 ML-Enhanced IC Reverse Engineering

As introduced in Sect. 13.3, reverse engineering is one main category of approaches to detect hardware Trojans. One existing work [9] introduces their approach of utilizing ML-based classification techniques to detect HTs at the layout level. They break layout images into several nonoverlapping blocks and obtain area and centroid differences as the associated features for each block. Those features are leveraged to train SVM and K-means clustering models. With trained models, they can identify HTs in the target design using its features without knowledge about the golden design.

Another work [104] shows the application of ML-based techniques in image processing during RE. Accurate image processing can help understand functionality from the images, and thus, the potential vulnerabilities can be correctly identified in the following RE steps. In the typical RE flow, after obtaining the images of the design, the next step is to annotate the design to understand the functionality of the design from the images, which involves a lot of work of image processing. ML can be effectively utilized in this step, including denoising, segmentation, and vectorization. Thus, ML-enabled image analysis can promise high-quality RE for the target design.

ReIGNN [17] is an approach that helps accelerate the RE process, especially the schematic creation, organization, and analysis step. To identify vulnerabilities at the schematic level, it is important to reconstruct the netlist and then understand the functionality of the design, so that it can be compared to a golden model to check whether any vulnerability exists. To reconstruct the netlist, it is critical to separate the control logic and data path. In a sequential design, registers decide the current states and play a significant role in composing the control logic. Therefore, correctly identifying the control logic registers is important. Conventional approaches apply heuristic clustering methods, whose accuracy is very sensitive to the configuring parameter values. Unlike previous methods, ReIGNN converts the netlist into a graph and applies a GNN-based model to classify registers. ReIGNN provides better identification performance and prevents users from exploring the best configuration in a huge space.

13.4.2.2 Discussion and Future Directions

RE is an essential technique in the current design testing and verification processes. Within the IC RE flow, there is a significant reliance on image processing and

graph analysis, where ML excels. Existing ML-based approaches may facilitate RE techniques in annotation and netlist regeneration due to their efficiency in related tasks. Therefore, developing more advanced ML-enabled RE techniques is promising. Moreover, ML-based approaches can potentially enhance the performance of techniques that rely on RE, such as hardware Trojan detection. Exploring the integration of ML in specific scenarios alongside traditional RE techniques presents intriguing opportunities for advancement.

13.5 IC Identifiers

IC counterfeiting and overbuilding are typical threats that undermine the trustworthiness of hardware design. Among the defense techniques, IC watermarking and fingerprinting can track the source of piracy and help designers claim ownership. In this section, we focus on the physical unclonable function (PUF), widely used as a unique identifier due to its physical properties.

13.5.1 Physical Unclonable Function

A physical unclonable function (PUF) is often a crucial component of a secure hardware design. PUF technology plays a key role in enhancing the security of a growing number of embedded devices. A PUF takes advantage of the inherent manufacturing variations present in embedded devices [105]. It leverages these variations to generate a device-specific random response, known as a PUF response. This response exhibits unpredictability, making it difficult to predict or replicate, and therefore serves as a distinctive fingerprint for each device. The primary applications of this fingerprint are authentication and the derivation of a device-specific secret key. By utilizing the unique PUF response, the authenticity of a device can be verified, ensuring that it is genuine and not a counterfeit or tampered device. Additionally, the PUF response can be used as a basis for generating a secret key that is tied to the specific device. This key can then be used for secure operations, such as encryption and decryption, providing an additional layer of device-specific security.

A PUF can be conceptually understood as a circuit element that produces a response, denoted as "r," when a challenge, represented as "ch," is applied to it. Currently, a diverse range of PUF primitives exists, including SRAM PUFs [106, 107], arbiter PUFs (APUF) [108], lightweight secure PUFs (LSPUF) [109], bistable ring PUF [110], multiplexer PUF (MPUF) [111], and many others. Ongoing research in this domain holds great potential for further advancements and innovations.

Table 13.2 ML-based modeling attacks on PUF designs

Attack on PUF	PUF Type	Contribution
[112]	Lightweight PUFs	This work exhibits that CRPs of lightweight PUFs can be learned and predicted by ML-based modeling attacks
[113]	XOR arbiter PUF	This paper proves that nonlinear XOR arbiter PUF is not secure
[114]	BR-PUF	The behaviors of PUFs unknown mathematical models can also be captured by ML attacks
[115]	Arbiter PUFs	The proposed learning algorithm can model CRPS of arbiter PUFs with prescribed accuracy and confidence with polynomial time complexity
[116]	Arbiter PUFs	This work improves learning processes and provides different speed options

13.5.2 ML-Based Attacks on PUF (Table 13.2)

The PUF circuit itself can be viewed as a function, and it has been shown through the universal approximation theorem that multilayer perceptrons (MLPs) have the ability to learn any input-output function. Consequently, PUFs inherently possess vulnerabilities to attacks leveraging deep learning and machine-learning techniques. In particular, machine learning-based modeling attacks are widely recognized as the most significant threat to PUF implementations [112, 117]. Numerous studies have demonstrated the effectiveness of machine learning-based attacks against PUFs, prompting extensive research efforts to develop newer attack methods. These attacks typically involve providing the adversary with access to a small portion of the challenge-response pairs (CRPs) dataset associated with a specific PUF instance. With this limited data, the adversary constructs a computational model capable of accurately predicting the PUF's response to any given challenge, achieving a high probability of success. The nonlinearity from the ReLU activation function also plays an important role in the attacks. Several ML or deep learning techniques have been commonly employed to launch attacks on PUFs, including support vector machines (SVMs), logistic regressions (LRs), and MLPs. These techniques have demonstrated their superior performance in attacking various PUF variants including various APUFs, bistable ring PUFs, and LSPUFs [108, 112, 118–121].

The paper [112] focuses on the security analysis from the perspective of ML on composite PUFs and multi-bit output lightweight secure PUFs (LSPUFs). First, the paper introduces a cryptanalysis attack that shows that composite PUFs with multiple levels of PUFs are not secure. Additionally, a machine-learning attack is developed for composite PUFs, which reduces the attack complexity. The paper highlights that the security evaluation of composite PUFs was not provided in the original proposal. Next, the paper discusses an attack on multi-bit output LSPUFs. LSPUFs rely on the design properties of input and output networks, as well as the primitive PUF instances used. The paper presents a new cryptanalysis attack that is effective for any multi-bit output LSPUF variants with practical time and

data complexity. The attack demonstrates that multi-bit output LSPUFs are not secure. The paper validates the proposed attacks through experimental results on field programmable gate array (FPGA) implemented composite PUFs and multi-bit output LSPUFs. The experiments demonstrate the effectiveness of the attacks in the best-case scenario. This paper provides a comprehensive analysis of the security of composite PUFs and multi-bit output LSPUFs, highlighting their vulnerabilities and the need for careful consideration of security trade-offs in PUF design.

A machine learning-based attack on arbiter physical unclonable functions (PUFs) is presented in [113]. PUFs are used for authentication and fingerprinting purposes, but it has been shown that arbiter PUFs are susceptible to various types of attacks. ML techniques have been successful in breaking the security of arbiter PUFs by mathematically cloning them. The attack proposes a new framework for learning XOR PUFs in the presence of noisy responses. It provides a theoretical limit for ML techniques to learn XOR PUFs based on the number of arbiter PUF stages and the number of chains. It also addresses the issue of noisy responses in XOR PUFs. It introduces a noise model that accurately represents the noise observed in practice. Unlike previous approaches that rely on majority voting to deal with noisy responses, this framework does not require majority voting and can still learn XOR PUFs efficiently. The authors emphasize that the security of XOR PUFs is limited by technological constraints, such as the number of chains and the silicon area required for construction. They argue that protocols relying solely on XOR PUFs cannot be considered perfectly secure. Overall, this attack highlights the limitations and vulnerabilities of arbiter PUFs. It provides insights into the design and manufacturing of delay-based PUFs and contributes to the understanding of ML attacks on PUFs.

Most machine-learning attacks on PUFs assume that a mathematical model of the PUF functionality is known. The paper [114] addresses the scenario where the mathematical model is unknown and proposes a provable framework for machine-learning attacks against PUFs. The authors focus on the bistable ring PUF (BR-PUF) family, which is a prime example of a PUF with an unknown mathematical model. The paper presents a novel probably approximately correct learning framework that can learn the challenge-response behavior of BR-PUFs without relying on a mathematical model. The authors conduct extensive experiments on BR-PUFs implemented on FPGA to evaluate the effectiveness of their attack. The experimental results confirm the applicability and effectiveness of their machine-learning algorithm. It also discusses the existence of influential bits in PUFs and provides a constant upper bound on the number of influential bits. This result is based on empirical evidence and theoretical insights.

PAC learning attacks [115] is a learning algorithm that can model the challenge-response behavior of an arbiter PUF with prescribed levels of accuracy and confidence. The algorithm is based on a new mathematical representation of arbiter PUFs, which allows for the discretization of real-valued delay values. By mapping the delay values to a finite set of integer values, the algorithm can effectively learn the behavior of the arbiter PUF. The paper proves that the maximum number of CRPs required for the attack is polynomial in the length of the arbiter PUF. The

experiments also evaluate the time complexity of the learning algorithm and show that it is polynomial in the length of the arbiter PUF, the maximum variation of delays, and the desired levels of accuracy and confidence.

Active learning attack is introduced in [116] and it aims to improve the learning process of PUFs, specifically the arbiter PUF. The researchers present two variations of the active learning technique: fast learning and slow learning. In fast learning, the goal is to quickly learn the PUF model with a small number of challenges. The technique selects challenges that are close to the current model of the hyperplane separating the PUF responses. This approach outperforms random challenge selection and achieves high accuracy with only 350 challenges for a 64-stage arbiter PUF. In slow learning, the objective is to generate a set of challenges that yield poor generalization errors, making it difficult for an attacker to accurately clone the PUF. The technique uses an initial set of challenges and a parameter k to select challenges that lie at different distances from the hyperplane. By tuning k, different learning speeds can be achieved. Slow learning with a high value of k results in challenges that are far from the hyperplane and statistically uncorrelated, making it harder to learn the PUF. The paper provides numerical results comparing the performance of the active learning technique with other learning algorithms such as LR, SVM, AdaBoost, and neural networks. The results demonstrate the effectiveness of the active learning technique in both noiseless and noisy scenarios, outperforming other methods in terms of prediction error rate and accuracy. The active learning technique offers a powerful approach to improve the learning process of arbiter PUFs, enabling faster and more accurate learning while also providing options for slower and more adversarial learning.

13.5.3 Defenses Countering ML-Based Attacks on PUF

In response, various defense mechanisms have been proposed to counter ML attacks. These defenses primarily focus on enhancing the nonlinearity of the PUF structure and challenge-response mapping [122–124]. These defense strategies aim to increase the complexity and nonlinearity of the PUF's behavior, making it more challenging for ML-based attacks to accurately model the PUF and predict its responses.

The work [118] introduces a defense mechanism called the interpose physical unclonable function (iPUF) to protect against machine-learning attacks on XOR arbiter PUFs (APUF). The iPUF is designed to be resilient against both classical machine-learning attacks and reliability-based machine-learning attacks. The defense is based on the concept of interposing a bit in the middle of the XOR APUF structure. This interposed bit disrupts the correlation between the challenge and response bits, making it difficult for machine-learning attacks to accurately model the PUF. The paper provides theoretical analysis and empirical results to demonstrate the effectiveness of the iPUF defense. To evaluate the security of the iPUF, the iPUF is compared with the XOR APUF using different machine-learning

attacks. The results show that the iPUF is more resilient to attacks compared to the XOR APUF. Classical machine-learning attacks, such as LR and deep neural networks, are less efficient on the iPUF. The paper further discusses the reliability-based machine-learning attacks and proposes an enhanced reliability-based CMA-ES attack. The results demonstrate that the enhanced attack using the proposed reliability model outperforms the original attack. The iPUF is shown to have higher modeling accuracy and a greater frequency of correct models compared to the XOR APUF. The iPUF defense provides a practical solution to enhance the security of XOR APUFs against machine-learning attacks. The interposed bit disrupts the correlation between challenge and response bits, making it difficult for attackers to accurately model the PUF. The empirical results and theoretical analysis support the effectiveness of the iPUF defense, making it a promising approach for securing PUFs against machine-learning attacks.

A new lightweight PUF design called the multi-PUF (MPUF) is proposed in [124]. The MPUF design aims to enhance the security of PUFs against modeling attacks by utilizing Weak PUFs to obfuscate the challenges to a Strong PUF. The architecture of the MPUF design consists of an array of elementary 1-bit cells, each composed of a group of Weak PUFs and a Strong PUF. The proposed MPUF design demonstrates higher complexity compared to the conventional arbiter PUF, making it more difficult to attack. A mathematical model of the MPUF design is presented, showing the delay difference between the two delay paths in the APUF can be represented as the inner product of the parity vector and a constant vector. The mathematical model also shows that the actual input challenge to the APUF is obfuscated and masked, further increasing the complexity of the design. To evaluate the resistance of the MPUF design against modeling attacks, two machine learning-based attacks, LR and covariance matrix adaptation evolution strategies (CMA-ES), are used. The experimental results show that the proposed MPUF design achieves a lower prediction rate compared to the conventional Arbiter PUF, indicating its improved resistance to modeling attacks. The proposed MPUF design demonstrates higher complexity, improved resistance to modeling attacks, and good performance in terms of uniqueness and uniformity.

Another representative ML defensive PUF is proposed in [125]. A configurable tristate PUF against machine-learning attacks is introduced for IoT security. The CT PUF is designed to provide secure device authentication by generating unique responses based on challenge inputs. It incorporates obfuscation mechanisms and employs a bitwise XOR operation to hide the mapping relationship between challenges and responses. Experimental results show the resistance of CT PUF against empirical ML attack methods. The results show that CT PUF is resilient to these ML attacks. It also discusses the resistance of CT PUF to provable ML attack methods, specifically the PAC learning method. It highlights the challenges in breaking CT PUF using this method due to the obfuscation mechanisms and the difficulty in approximating CT PUF with a known Boolean function. Furthermore, the paper also presents the CT PUF-based authentication protocol, which includes an enrollment phase and an authentication phase. The CT PUF defense demonstrates its effectiveness in providing secure device authentication and resistance against

various machine-learning attack methods. The experimental and mathematical analysis results support the claim that CT PUF is a robust defense mechanism for IoT security.

13.5.4 Discussion and Future Directions

The aforementioned studies demonstrate the efficacy of ML techniques in PUF modeling attacks, where the primary challenge is to accurately learn the mathematical model governing the challenge and response behaviors of PUFs. ML offers distinct advantages in handling complex, high-dimensional models, which explains its success in PUF modeling attacks. Given the critical role of PUFs in ensuring hardware trustworthiness, it is foreseeable that further research will delve into assessing the security of diverse types of PUFs under various scenarios and with attack assumptions. This will also drive the development of more resilient and secure PUF designs, thereby supporting the overall reliability of hardware systems.

Beyond modeling attacks, ML can be used in other aspects of PUF design. For example, a recent work [126] showcases the potential of ML in enhancing the authentication phase of PUFs. They build a CNN model to help improve noise tolerance of the authentication step without any more profound knowledge of the PUF structure and characteristics. The prospect of further research exploring the application of ML from the defender's perspective in PUF design or authentication processes would be intriguing. Such investigations could yield innovative approaches further to enhance the security and reliability of PUF designs.

13.6 Conclusion

In this chapter, we reviewed existing design methods that incorporate ML techniques to enhance IC security throughout the supply chain, from design, testing, and vulnerability detection, to IC identification. We discussed how IC security can benefit from ML techniques and their integration in conventional design flows. Specifically, in an integrated approach, conventional IC design methods can be used to (a) generate training datasets for building accurate ML models, and (b) as a domain-specific formal foundation to rectify the outcomes of ML models and improve on their accuracy and robustness. On the other hand, ML can help with (a) capturing important features of complex systems, as in modeling attacks on PUFs; (b) efficiently guiding decision-making toward promising directions, as in test pattern generation for hardware fuzzing; (c) efficiently detecting recurrent structural patterns in circuits, for example, in the context of IC reverse engineering; and (d) efficiently exploring large design spaces, as discussed in the context of hardware IP protection and confidentiality.

Emerging ML techniques bring further opportunities and challenges. LLMs have already demonstrated remarkable capabilities in providing innovative design solutions [127, 128]. They can rapidly generate designs in hardware description languages when the design specifications are provided in natural languages. The models can provide functionally correct codes for common designs, and the designs can be further refined with feedback either from human engineers or from the testing results. However, as for most ML techniques, the resulting models have limited interpretability and rely heavily on data. Their output still requires careful verification and validation. Overall, as hardware designs become increasingly more complex, the principled combination of ML techniques with conventional design methodologies holds the promise of ushering in a new era for secure and trustworthy hardware design.

References

1. Hassija, V., et al. (2020). A survey on supply chain security: Application areas, security threats, and solution architectures. *IEEE IoT Journal, 8*, 6222–6246.
2. Guin, U., et al. (2014). Counterfeit integrated circuits: A rising threat in the global semiconductor supply chain. *Proceedings of the IEEE, 102*, 1207–1228.
3. Liu, B., & Qu, G. (2016). VLSI supply chain security risks and mitigation techniques: A survey. *Integration, 55*, 438–448.
4. Rostami, M., Koushanfar, F., & Karri, R. (2014). A primer on hardware security: Models, methods, and metrics. *Proceedings of the IEEE, 102*, 1283–1295.
5. Kamali, H. M., et al. (2022). Advances in logic locking: Past, present, and prospects. *Cryptology ePrint Archive*.
6. Bhasin, S., & Regazzoni, F. (2015). *A survey on hardware trojan detection techniques* (pp. 2021–2024). IEEE.
7. Trippel, T., et al. (2022). *Fuzzing hardware like software* (pp. 3237–3254).
8. Cruz, J., et al. (2018). *Hardware Trojan detection using ATPG and model checking* (pp. 91–96). IEEE.
9. Bao, C., et al. (2016). On reverse engineering-based hardware Trojan detection. *TCAD, 35*, 49–57.
10. Dey, K., et al. (2021). *PUF based hardware security: A review* (pp. 1–6). IEEE.
11. Chen, X., Qu, G., & Cui, A. (2017). *Practical IP watermarking and fingerprinting methods for ASIC designs* (pp. 1–4). IEEE.
12. Hu, Y., et al. (2021). *Risk-aware cost-effective design methodology for integrated circuit locking* (pp. 1182–1185). IEEE.
13. Beerel, P., Georgiou, M., Hamlin, B., Malozemoff, A. J., & Nuzzo, P. (2022). Towards a formal treatment of logic locking. *Cryptology ePrint Archive, 2022*, 92.
14. Li, J., Zhao, B., & Zhang, C. (2018). Fuzzing: a survey. *Cybersecurity, 1*, 1–13.
15. Hu, W., et al. (2020). An overview of hardware security and trust: Threats, countermea- sures, and design tools. *TCAD, 40*, 1010–1038.
16. Sisejkovic, D., et al. (2021). Challenging the security of logic locking schemes in the era of deep learning: A neuroevolutionary approach. *JETC, 17*, 30.
17. Chowdhury, S. D., Yang, K., & Nuzzo, P. (2021). *ReIGNN: State register identification using graph neural networks for circuit reverse engineering* (pp. 1–9).
18. Pan, I., et al. (2022). Data-centric engineering: Integrating simulation, machine learning and statistics. Challenges and opportunities. *CES, 249*, 117271.

19. O'Mahony, N., et al. (2020). *Deep learning vs. traditional computer vision* (pp. 128–144). Springer.
20. Hu, Y., et al. (2023). On the security of sequential logic locking against oracle-guided attacks. *TCAD, 42*, 3628.
21. Zhang, Y., et al. (2022). *Trilock: IC protection with tunable corruptibility and resilience to SAT and removal attacks* (pp. 1329–1334). IEEE.
22. Hu, Y., et al. (2021). *Fun-SAT: Functional corruptibility-guided SAT-based attack on sequential logic encryption* (pp. 281–291).
23. Hu, Y., et al. (2020). *SANSCrypt: A sporadic-authentication-based sequential logic encryption scheme* (pp. 129–134).
24. Chowdhury, S. D., et al. (2021). *Enhancing SAT-attack resiliency and cost-effectiveness of reconfigurable-logic-based circuit obfuscation* (pp. 1–5). IEEE.
25. Hu, Y., et al. (2019). *Security-driven metrics and models for efficient evaluation of logic encryption schemes* (pp. 1–5).
26. Subramanyan, P., Ray, S., & Malik, S. (2015). *Evaluating the security of logic encryption algorithms* (pp. 137–143). IEEE.
27. Yasin, M., et al. (2015). On improving the security of logic locking. *TCAD, 35*, 1411–1424.
28. Xu, X., et al. (2017). *Novel bypass attack and BDD-based tradeoff analysis against all known logic locking attacks* (pp. 189–210). Springer.
29. Yasin, M., et al. (2017). Removal attacks on logic locking and camouflaging techniques. *Transactions on Emerging Topics in Computing, 8*, 517–532.
30. Sirone, D., & Subramanyan, P. (2020). Functional analysis attacks on logic locking. *Transactions on Information Forensics and Security, 15*, 2514–2527.
31. Chen, D., Lin, C., & Beerel, P. A. (2021). *GF-Flush: A GF(2) algebraic attack on secure scan chains*. IEEE.
32. Alrahis, L., et al. (2022). OMLA: An oracle-less machine learning-based attack on logic locking. *Transactions on Circuits and Systems II: Express Briefs, 69*, 1602–1606.
33. Alrahis, L., et al. (2021). *GNNUnlock: Graph neural networks-based oracle-less unlocking scheme for provably secure logic locking* (pp. 780–785).
34. Tehranipoor, F., et al. (2019). *Deep RNN-oriented paradigm shift through BOCANet: Broken obfuscated circuit attack* (pp. 335–338).
35. Chen, H., et al. (2019). *GenUnlock: An automated genetic algorithm framework for unlocking logic encryption* (pp. 1–8).
36. Chakraborty, P., Cruz, J., & Bhunia, S. (2019). *SURF: Joint structural functional attack on logic locking* (pp. 181–190).
37. Chakraborty, P., Cruz, J., & Bhunia, S. S. A. I. L. (2018). *Machine learning guided structural analysis attack on hardware obfuscation* (pp. 56–61).
38. Alaql, A., Forte, D., & Bhunia, S. (2019). *Sweep to the secret: A constant propagation attack on logic locking* (pp. 1–6).
39. Azar, K. Z., et al. (2020). *NNgSAT: Neural network guided SAT attack on logic locked complex structures* (pp. 1–9).
40. Chen, D., et al. (2023). Unraveling latch locking using machine learning, boolean analysis, and ILP. *arXiv preprint arXiv:2305.00107*.
41. Alrahis, L., et al. (2021). UNSAIL: Thwarting oracle-less machine learning attacks on logic locking. *Transactions on Information Forensics and Security, 16*, 2508–2523.
42. Hu, Y., et al. (2024). DECOR: Enhancing logic locking against machine learning-based attacks. *arXiv preprint arXiv:2403.01789*.
43. Lu, Y., et al. (2023). RTLLM: An open-source benchmark for design rtl generation with large language model. *arXiv preprint arXiv:2308.05345*.
44. Thakur, S., et al. (2023). *Benchmarking large language models for automated verilog rtl code generation* (pp. 1–6). IEEE.
45. Ahmed, B., et al. (2024). *Seemless: Security evaluation of logic locking using machine learning oriented estimation* (pp. 489–494).

46. Huang, Z., et al. (2020). A survey on machine learning against hardware Trojan attacks: Recent advances and challenges. *IEEE Access, 8*, 10796–10826.
47. Liakos, K. G., et al. (2019). *Machine learning for hardware Trojan detection: A review* (pp. 1–6).
48. Patnaik, S., et al. (2022). *Reinforcement learning for hardware security: Opportunities, developments, and challenges* (pp. 217–218).
49. Liakos, K. G., et al. (2020). Conventional and machine learning approaches as countermeasures against hardware Trojan attacks. *Microprocessors and Microsystems, 79*, 103295.
50. Chen, H., et al. (2023). AdaTest: Reinforcement learning and adaptive sampling for on-chip hardware trojan detection. *TECS, 22*, 1–23.
51. Pan, Z., & Mishra, P. (2021). *Automated test generation for hardware Trojan detection using reinforcement learning* (pp. 408–413).
52. Gohil, V., et al. (2022). *DETERRENT: Detecting trojans using reinforcement learning* (pp. 697–702).
53. Yang, S., Chakraborty, P., & Bhunia, S. (2021). *Side-channel analysis for hardware Trojan detection using machine learning* (pp. 1–6).
54. Kulkarni, A., et al. (2016). Real-time anomaly detection framework for many-core router through machine-learning techniques. *JETC, 13*, 1–22.
55. Kulkarni, A., Pino, Y., & Mohsenin, T. (2016). *SVM-based real-time hardware Trojan detection for many-core platform* (pp. 362–367).
56. Xue, M., Wang, J., & Hu, A. (2016). *An enhanced classification-based golden chips-free hardware Trojan detection technique* (pp. 1–6).
57. Liu, Y., Huang, K., & Makris, Y. (2014). *Hardware Trojan detection through golden chip-free statistical side-channel fingerprinting* (pp. 1–6).
58. Salmani, H. (2017). COTD: Reference-free hardware Trojan detection and recovery based on controllability and observability in gate-level netlist. *Transactions on Information Forensics and Security, 12*, 338–350.
59. Cakır, B., & Malik, S. (2015). *Hardware Trojan detection for gate-level ICs using signal correlation-based clustering* (pp. 471–476).
60. Hasegawa, K., et al. (2016). *Hardware trojans classification for gate-level netlists based on machine learning* (pp. 203–206). IEEE.
61. Hasegawa, K., et al. (2017). *Trojan-feature extraction at gate-level netlists and its application to hardware-trojan detection using random forest classifier* (pp. 1–4). IEEE.
62. Yasaei, R., Yu, S.-Y., & Al Faruque, M. A. (2021). *Gnn4tj: Graph neural networks for hardware trojan detection at register transfer level* (pp. 1504–1509).
63. Yasaei, R., Chen, L., Yu, S.-Y., & Faruque, M. A. A. (2022). *Hardware trojan detection using graph neural networks* (pp. 1–1). TCAD.
64. Cruz, J. (2022). et al, Automatic hardware trojan insertion using machine learning. *arXiv preprint arXiv:2204.08580*.
65. Saha, D., et al. (2023). Llm for soc security: A paradigm shift. *arXiv preprint arXiv:2310.06046*.
66. Chen, C., et al. (2023). *HyPFuzz: Formal-assisted processor fuzzing* (pp. 1361–1378).
67. Yu, S.-Y., Yasaei, R., Zhou, Q., Nguyen, T., & Al Faruque, M. A. (2021). *Hw2vec: A graph learning tool for automating hardware security* (pp. 13–23). IEEE.
68. Louridas, P. (2006). Static code analysis. *IEEE Software, 23*, 58–61.
69. Gomes, I., et al. (2009). An overview on the static code analysis approach in software development. *Faculdade de Engenharia da Universidade do Porto*.
70. Dahse, J., & Holz, T. (2014). *Simulation of built-in php features for precise static code analysis* (Vol. 14, pp. 23–26).
71. Fell, J. (2017). A review of fuzzing tools and methods. *PenTest Magazine*.
72. Böttinger, K., et al. (2018). *Deep reinforcement fuzzing* (pp. 116–122). IEEE.
73. Wang, Y., Jia, P., Liu, L., Huang, C., & Liu, Z. (2020). A systematic review of fuzzing based on machine learning techniques. *PLoS One, 15*, e0237749.
74. Foster, J. C. (2005). *Buffer overflow attacks*.

75. Nishimura, R., et al. (2016). *Implementation of the can-fd protocol in the fuzzing tool bestorm* (pp. 1–6). IEEE.
76. Wong, C. W. K. (2022). *American fuzzy lop (afl) fuzzer*.
77. Prasad, M. R., Chong, P., & Keutzer, K. (1999). *Why is atpg easy?* (pp. 22–28).
78. Cha, S. K., Woo, M., & Brumley, D. (2015). *Program-adaptive mutational fuzzing* (pp. 725–741). IEEE.
79. Smetsers, R., Moerman, J., Janssen, M., & Verwer, S. (2016). Complementing model learning with mutation-based fuzzing. *arXiv preprint arXiv:1611.02429*.
80. Godefroid, P., et al. (2008). *Grammar-based whitebox fuzzing* (pp. 206–215).
81. Liu, J., et al. (2023). Generation-based differential fuzzing for deep learning libraries. *ACM Transactions on Software Engineering and Methodology, 33*, 1–28.
82. Li, Y., et al. (2019). V-fuzz: Vulnerability-oriented evolutionary fuzzing. *arXiv preprint arXiv:1901.01142*.
83. Lou, B., & Song, J. (2020). A study on using code coverage information extracted from binary to guide fuzzing. *IJCSS, 14*, 200–210.
84. How to cyber security: Containerizing fuzzing targets. (2021). https://www.synopsys.com/blogs/software-security/effective-fuzz-testing-containerization/
85. Zhang, B., Ye, J., Bi, X., Feng, C., & Tang, C. (2018). Ffuzz: Towards full system high coverage fuzz testing on binary executables. *PLoS One, 13*, e0196733.
86. Serebryany, K., & Iskhodzhanov, T. (2009). *Threadsanitizer: Data race detection in practice* (pp. 62–71).
87. Serebryany, K., Bruening, D., Potapenko, A., & Vyukov, D. (2012). *AddressSanitizer: A fast address sanity checker* (pp. 309–318).
88. Yang, G., Feng, C., & Tang, C. (2017). *Static analysis assisted vulnerability-oriented evolutionary fuzzing* (Vol. 1820). AIP Publishing.
89. Shields, P. (2023). *Hybrid testing: Combining static analysis and directed fuzzing*. Ph.D. thesis,. Massachusetts Institute of Technology.
90. Nichols, N., et al. (2017). Faster fuzzing: Reinitialization with deep neural models. *arXiv preprint arXiv:1711.02807*.
91. Lyu, C., et al. (2018). SmartSeed: Smart seed generation for efficient fuzzing. *arXiv preprint arXiv:1807.02606*.
92. Godefroid, P., Peleg, H., & Singh, R. (2017). *Learn&Fuzz: Machine learning for input fuzzing, 50–59*. IEEE.
93. Cheng, L., et al. (2019). *Optimizing seed inputs in fuzzing with machine learning* (pp. 244–245). IEEE.
94. Becker, S., Abdelnur, H., State, R., & Engel, T. (2010). *An autonomic testing framework for ipv6 configuration protocols* (pp. 65–76). Springer.
95. Fang, K., & Yan, G. (2018). *Emulation-instrumented fuzz testing of 4g/lte android mobile devices guided by reinforcement learning* (pp. 20–40). Springer.
96. Drozd, W., & Wagner, M. D. (2018). Fuzzergym: A competitive framework for fuzzing and learning. *arXiv preprint arXiv:1807.07490*.
97. Li, X., Liu, X., Chen, L., Prajapati, R., & Wu, D. (2022). *Fuzzboost: Reinforcement compiler fuzzing* (pp. 359–375). Springer.
98. Ouyang, Y., Wu, Z., Mu, Q., & Wang, Q. (2014). *Leveraging markov chain and optimal mutation strategy for smart fuzzing*. IET.
99. Wang, Y., Wu, Z., Wei, Q., & Wang, Q. (2019). Neufuzz: Efficient fuzzing with deep neural network. *IEEE Access, 7*, 36340–36352.
100. Meade, T., Zhang, S., & Jin, Y. (2016). *Netlist reverse engineering for high-level functionality reconstruction* (pp. 655–660).
101. Meade, T., et al. (2016). *Gate-level netlist reverse engineering for hardware security: Control logic register identification* (pp. 1334–1337).
102. Brunner, M., Baehr, J., & Sigl, G. (2019). *Improving on state register identification in sequential hardware reverse engineering* (pp. 151–160).

103. Geist, J., et al. (2020). *RELIC-FUN: Logic identification through functional signal comparisons* (pp. 1–6).
104. Botero, U. J., et al. (2021). Hardware trust and assurance through reverse engineering: A tutorial and outlook from image analysis and machine learning perspectives. *JETC, 17*, 1–53.
105. Gassend, B., et al. (2002). *Silicon physical random functions* (pp. 148–160).
106. Guajardo, J., et al. (2007). *FPGA intrinsic PUFs and their use for IP protection* (pp. 63–80). Springer.
107. Chen, B., et al. (2017). *A robust SRAM-PUF key generation scheme based on polar codes* (pp. 1–6). IEEE.
108. Lim, D., et al. (2005). Extracting secret keys from integrated circuits. *Transactions on Very Large Scale Integration (VLSI) Systems, 13*, 1200–1205.
109. Majzoobi, M., et al. (2008). *Testing techniques for hardware security* (pp. 1–10). IEEE.
110. Chen, Q., et al. (2011). *The bistable ring PUF: A new architecture for strong physical unclonable functions* (pp. 134–141). IEEE.
111. Sahoo, D. P., et al. (2017). A multiplexer-based arbiter PUF composition with enhanced reliability and security. *Transactions on Computers, 67*, 403–417.
112. Sahoo, D. P., et al. (2015). A case of lightweight PUF constructions: Cryptanalysis and machine learning attacks. *TCAD, 34*, 1334–1343.
113. Ganji, F., Tajik, S., & Seifert, J.-P. (2015). *Why attackers win: On the learnability of xor arbiter pufs* (pp. 22–39). Springer.
114. Ganji, F., et al. (2016). *Strong machine learning attack against pufs with no mathematical model* (pp. 391–411). Springer.
115. Ganji, F., et al. (2016). Pac learning of arbiter pufs. *Journal of Cryptographic Engineering, 6*, 249–258.
116. Dumoulin, V., Rao, W., & Devroye, N. (2023). Active learning for fast and slow modeling attacks on arbiter pufs. *arXiv preprint arXiv:2308.13645*.
117. Nguyen, P. H., et al. (2016). Security analysis of arbiter PUF and its lightweight compositions under predictability test. *TODAES, 22*, 1–28.
118. Nguyen, P. H., et al. (2018). The interpose PUF: Secure PUF design against state-of- the-art machine learning attacks. *Cryptology ePrint Archive*.
119. Ruhrmair, U., et al. (2010). *Modeling attacks on physical unclonable functions* (pp. 237–249).
120. Tobisch, J., & Becker, G. T. (2015). *On the scaling of machine learning attacks on PUFs with application to noise bifurcation* (pp. 17–31). Springer.
121. Schuster, D., & Hesselbarth, R. (2014). *Evaluation of bistable ring PUFs using single layer neural networks* (pp. 101–109). Springer.
122. Awano, H., & Sato, T. (2018). *Ising-PUF: A machine learning attack resistant PUF featuring lattice like arrangement of arbiter-PUFs* (pp. 1447–1452). IEEE.
123. Zalivaka, S. S., Ivaniuk, A. A., & Chang, C.-H. (2017). *Low-cost fortification of arbiter PUF against modeling attack* (pp. 1–4). IEEE.
124. Ma, Q., et al. (2018). *A machine learning attack resistant multi-PUF design on FPGA, 97–104*. IEEE.
125. Zhang, J., et al. (2021). CT PUF: Configurable tristate PUF against machine learning attacks for iot security. *IEEE IoT Journal, 9*, 14452–14462.
126. Millwood, O., et al. (2023). Puf-phenotype: A robust and noise-resilient approach to aid group-based authentication with dram-pufs using machine learning. *IEEE Transactions on Information Forensics and Security, 18*, 2451.
127. Blocklove, J., Garg, S., Karri, R., & Pearce, H. (2023). *Chip-chat: Challenges and opportunities in conversational hardware design* (pp. 1–6). IEEE.
128. Thakur, S., et al. (2023). Autochip: Automating hdl generation using llm feedback. *arXiv preprint arXiv:2311.04887*.